# KAVACH AI Project Report

## Website features architecture and operation

Implementation reviewed 7 November 2026

### Executive summary

KAVACH AI is a local document intelligence website for reading, reviewing, searching, and discussing user-supplied documents. A user uploads a file, chooses an analysis mode and domain profile, and receives results that can be inspected alongside the source. The application keeps document records, analysis history, review decisions, batch progress, and generated reports on the configured local backend.

The website supports risk review, document summaries, literal field extraction, a searchable Knowledge Base, document chat with citations, and highlighted evidence. Several files can be uploaded and analyzed as a batch. An optional Python Sandbox runs code that the user explicitly submits, separately from document analysis.

The project combines a Next.js frontend, a FastAPI backend, SQLite persistence, local document extraction and OCR, and configured local AI models. Its main use case is helping a person understand internal reports, invoices, contracts, technical records, study material, and administrative files while retaining a path back to the evidence.

Source verification checks citation support. It does not establish that a document or an AI answer is factually correct. The current application is designed for a trusted local workstation; broader deployment needs additional access, network, and storage controls.

### How to read this report

- Pages 2 to 4 explain the purpose, supported inputs, and every website page.
- Pages 5 and 6 explain the architecture and analysis pipeline.
- Pages 7 to 9 explain evidence highlighting, batches, and document chat.
- Pages 10 to 12 cover storage, APIs, setup, operating limits, and security.
- Pages 13 and 14 provide a demonstration plan, recorded validation, a glossary, and implementation references.

### Report scope

This report describes the implementation in the project workspace, including the recently added batch, chat, and evidence features. Examples are illustrative unless identified as recorded test fixtures. It is a product and technical explanation, not an independent security audit or an accuracy benchmark.

*Source basis S1 S2 S11*

# Project purpose and scope

## The problem being addressed

Useful information is often spread across long documents, scanned pages, and separate records. A reviewer may need to identify dates, amounts, obligations, or risk-related statements, then explain where those details came from. Sending internal files to an external AI service may also conflict with the user's intended handling of those documents.

KAVACH AI brings file storage, local processing, source inspection, and report generation into one workflow. The user remains responsible for checking results and making decisions. The project does not claim that every possible risk can be detected or that model output is always correct.

## Main objectives

- Read supported digital documents and use local text recognition when scans need it.
- Provide useful analysis choices without requiring users to operate backend tools directly.
- Keep findings and fields linked to source excerpts and locations.
- Make agent actions, processing progress, errors, and review requirements visible.
- Preserve analysis history and generate downloadable files.
- Support several documents through batches, retrieval, and document chat.

## Supported inputs and analysis modes

The upload control accepts PDF, Word DOCX, plain text TXT, CSV, PNG, JPG, JPEG, and Python PY source files. The frontend allows up to 25 MB per file; the backend also enforces its configured upload limit. A Python upload is stored as a document and is not automatically executed.

| Mode | What it does | What the user should check |
|---|---|---|
| Risk review | Matches the selected profile's risk vocabulary against source text and records supporting excerpts. Local model reasoning can add advisory commentary. | Whether the matched passage is relevant and actually describes an issue in context. |
| Summary | Uses the configured local text or vision model to summarize or describe the input. | The original document and any omissions or unsupported model statements. |
| Extract fields | Finds literal dates, amounts, contacts, owners, reference identifiers, and measurements with source locations. | That each extracted value and its meaning match the original. |

## Intended users and document contexts

Eight built-in profiles cover everyday documents, industrial engineering, business and projects, education and research, finance and procurement, contracts and policies, healthcare administration, and people and HR. Profiles guide document context and review vocabulary. These profiles do not make the application a qualified legal, medical, financial, or engineering decision maker.

*Source basis S1 S3 S7 S12*

# Website pages for document work

## Overview and landing page

The home page introduces the project, explains the analysis modes, shows a sample invoice workflow, and provides a guide to the workspace tools. Profile cards open Workbench with a suggested mode and request. The workspace snapshot loads actual stored-document and recorded-run counts and links to recent activity. Missing backend data is shown as unavailable rather than replaced with sample statistics.

## Document Vault

The Vault is the starting point for source files. Users can upload one or more files, inspect per-file upload outcomes, preview supported originals, download them, and select documents for batch analysis. The table shows document metadata and available processing states. Single-document analysis remains available alongside batch processing.

Deleting a document requires confirmation in the website. Deletion removes the uploaded source and excludes it from future knowledge searches, while existing run history and reports remain. Deletion is blocked during active analysis, unfinished batches, and source-pinned chat requests. Retained history cannot open an original that has been deleted.

## Workbench

Workbench lets users select a stored source, domain profile, analysis mode, and editable request. The normal Analyze document action starts a persisted background job. The optional bounded planner uses a restricted set of actions to organize the task. A stage display exposes actual work such as extraction, retrieval, reasoning, verification, and report saving.

After completion, Workbench shows model output where applicable, source matches or extracted fields, review controls, and report downloads. A saved job link or browser refresh reconnects to the same recorded job rather than automatically creating another analysis.

## Knowledge Base

This page searches readable text from stored documents. It shows indexed sources, relevant passages, page information, and retrieval scores. Search results can open highlighted source previews. A retrieval score describes similarity in the configured index; it is not a factual confidence score.

## Document Chat

Users select up to 20 stored documents and ask questions or follow-ups. Retrieval and generation are scoped to that selection. A displayed answer includes known source markers that open the supporting passage. Missing evidence or invalid citation markers produce an insufficient-evidence response. Chat history stays in the current browser tab, clears on refresh, and resets when the selected sources change.

*Source basis S1 S2 S4 S5 S6*

# Website pages for review and operation

## Agent Runs

Agent Runs lists recorded analyses and their statuses. Opening a run exposes its execution trace, findings, source excerpts, available model output, and generated files. Evidence buttons open the highlighted source viewer. Findings held for review can be approved or rejected. Review decisions are saved and the current run state is returned when results are reopened.

## Deliverables

This page lists reports that actually exist for saved analyses and provides download links. Text and Markdown reports are standard outputs. Word DOCX, Excel XLSX, and PDF exports depend on the optional generation libraries available on the local installation. The website does not substitute a fabricated report when a file is missing.

## Model Router

Model Router explains the configured provider and local model assignments for reasoning, vision, and other tasks. Routing uses rules and configuration. A configured assignment does not prove that a model is loaded, healthy, accurate, or fast. Actual inference failures are surfaced when the model is called.

## Security Center

Security Center displays configured processing policy, code-execution readiness, and the local audit log. Audit events record activities such as uploads, analyses, reviews, and downloads. The page explains that application restrictions must be supported by host-level controls when the system is deployed more widely.

## System

System shows available local CPU, memory, GPU, VRAM, and service information. GPU history is built from actual measured samples when a supported reading exists. Missing measurements are labeled unavailable. Hardware readings help explain the workstation's state; they do not establish model quality or guarantee processing time.

## Python Sandbox

The sandbox is an optional, separate tool for Python code explicitly submitted by the user. Each execution uses a fresh restricted Linux container and returns standard output, standard error, exit status, and timing. The UI also shows whether the required local runtime is ready.

Uploading a document, analyzing an image, or starting the planner never automatically runs uploaded Python code. The sandbox has its own execution policy and runtime prerequisites. It can be disabled without removing the document-analysis workflow.

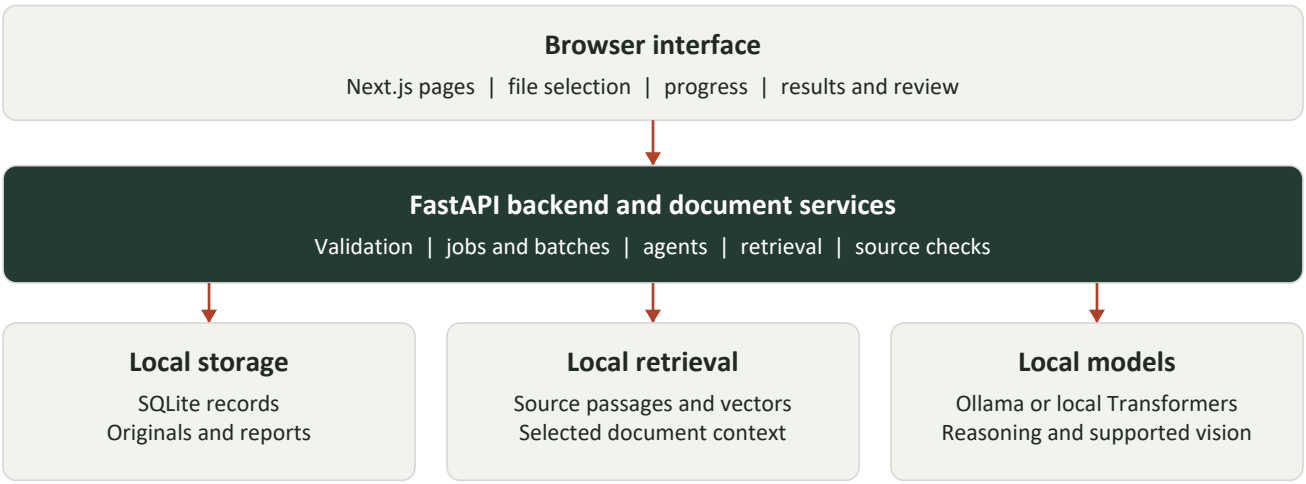
## Navigation and visual behavior

The frontend retains a shared workspace sidebar and top bar on internal pages. The landing page has its own introduction and responsive navigation. Existing fonts, paper tones, forest green, and orange accents provide a consistent identity. Keyboard focus indicators and reduced-motion preferences are preserved in the interface.

*Source basis S1 S2 S8 S9 S10*

# System architecture and technology

The browser sends requests to the local FastAPI backend, which coordinates validation, document services, jobs, agents, retrieval, inference, and reports. SQLite preserves records and processing state. Source files and generated reports remain in local directories.




## Main technology choices

| Layer | Implemented technology | Responsibility |
|---|---|---|
| Frontend | Next.js 14, React 18, TypeScript | Pages, navigation, forms, job progress, evidence views, chat, and downloads. |
| Styling | Tailwind CSS, custom CSS, Lucide icons | Responsive layouts, shared components, visual states, and accessible controls. |
| API | Python, FastAPI, Pydantic, Uvicorn | Request validation, local HTTP endpoints, service coordination, and error responses. |
| Persistence | SQLite and local directories | Document metadata, jobs, batches, runs, reviews, audit records, originals, and reports. |
| Document processing | pypdf, python-docx, Pillow, Tesseract through pytesseract, PyMuPDF | Text extraction, Word text and tables, OCR, and source-page rendering where installed. |
| Inference and retrieval | Ollama or local Transformers; lexical or provider embeddings | Local model generation and retrieval of relevant source passages. |

## Model roles and deployment boundary

Default Ollama assignments are Qwen3 8B for reasoning, LLaVA 7B for vision, and Llama 3.2 3B for other tasks and planner choices. Configuration can change these assignments. The Transformers adapter uses local model directories and is text-only. Literal field extraction and profile-term matching also use deterministic Python routines.

Normal runtime processing is designed around local services. Models and dependencies need to be provisioned before an offline installation is used. The application's airgapped setting is an application policy; network isolation still depends on deployment controls.

*Source basis S1 S8 S10 S12*

# How a document analysis works

The normal pipeline separates document preparation, inference, source checks, and report saving. The user can inspect progress rather than seeing only a final answer. A stage can be completed, skipped, warned, or failed according to the operation performed.

| Stage | Actual work performed |
|---|---|
| Check document | Confirm that the stored source and requested settings are usable. |
| Extract text and OCR | Read embedded or plain text. For sparse scanned PDF pages and raster images, attempt local OCR when available. Preserve page and location information. |
| Plan analysis | Apply provider policy, choose configured model roles, and prepare the request and document context. |
| Inspect images | For PNG, JPG, and JPEG analysis inputs, call the configured local vision model. Visual observations are kept separate from OCR-based fields. |
| Retrieve context | Split readable source text into passages, compute local vectors, and retrieve passages relevant to the task. |
| Analyze content | Run the selected mode. Summary uses inference; field extraction and risk-term review preserve literal source support. |
| Verify sources | Check excerpt and location support and determine which findings need human review. |
| Save results | Persist the run, trace, findings, evidence, and generated report files. |

## Specialized agents

The pipeline coordinates security, routing, document, vision, knowledge, reasoning, verification, and deliverable agents. Each component handles a defined responsibility. Their names describe software roles within the application, rather than independent people or unrestricted autonomous systems.

## Optional bounded planner

The planner is an alternative to following only the normal pipeline sequence. It requires a stored document, selects from allowlisted actions, and operates under step and loop limits. Unknown tools and invalid arguments are rejected. Its tools do not expose a general shell, arbitrary SQL, or unrestricted network access.

The planner can request document inspection, retrieval, reasoning, verification, and human review. The saved run records the chosen actions and stop reason. Findings that need approval remain held until the reviewer decides; a document with no matched findings can finish normally.

## Failure behavior

A failed summary or image inference returns an explicit error. For risk review, unavailable advisory reasoning can be recorded as a warning while supported source-term review continues. Neither a missing match nor an empty extraction establishes that a document has no problems.

*Source basis S3 S7 S8*

# Evidence highlighting and human review

## What evidence means in this project

A finding or field can include the source filename, page, quoted excerpt, extraction method, character span, and available OCR confidence and bounding boxes. These details allow a reviewer to inspect how a result relates to the uploaded document. Confidence and source verification describe extraction heuristics and citation support, not factual certainty.

## Opening a highlighted source

1. Select View highlighted source beside a finding, Knowledge Base result, or chat citation.
2. The backend resolves the stored document and source page or text section.
3. It checks that the excerpt matches the current extracted source text. Stored character spans are used when they still match; otherwise the exact excerpt is located in the source.
4. The viewer highlights the passage in the extracted text and shows surrounding context.
5. For PDF or raster images, the viewer also shows a rendered source page with available text or OCR locations overlaid. The original can be downloaded for inspection.

Word and plain-text sources use extracted text sections rather than a reproduction of the original printed layout. Word text and table content can be inspected, but formatting and embedded images should be checked in the original file.

## Source changes and missing originals

The viewer reports when the saved excerpt no longer matches the source or the original is unavailable. A deleted source is not silently rebound to a newly uploaded file with the same name. Older findings without a stored document link may require a new analysis before highlighting is available.

If a page cannot be rendered or its visual location cannot be found, verified extracted text may still be available. The UI distinguishes this fallback from a visual highlight on the original page.

## Human review decisions

Review requirements can arise from high-risk severity, absent or unknown evidence, low confidence relative to configuration, or missing verification. These checks flag findings for a person to inspect. The reviewer can approve or reject a held finding, and the application records the decision and updates the run state when all required decisions are complete.

Approval is a workflow decision, not an independent certification that the source or conclusion is correct. Model summaries and image observations remain advisory. Reviewers should read the actual passage and its context before relying on a result.

*Source basis S3 S6 S7 S10*

# Batch uploads and analysis

Batch processing applies shared analysis settings to several stored documents while keeping the result for each source separate. Uploading files and starting their analyses are distinct actions, so users can inspect upload outcomes before requesting processing.

## User workflow

1. Choose or drop up to 50 files in Document Vault. Each file is uploaded individually and receives a stored or failed outcome.
2. Successful uploads are selected in the Vault table. Users can adjust the selection or choose existing sources.
3. Choose the shared domain profile and analysis mode in the analysis settings.
4. Select Analyze selected. The backend admits the batch and saves its items before scheduling work.
5. Track waiting, running, completed, and failed items in the Batch analysis panel. Each source receives a job and, when processing creates one, a saved run.
6. Open an individual run or Workbench job to inspect progress, findings, review needs, and report downloads.

## Capacity and persistence

| Control | Current behavior |
| --- | --- |
| Upload selection | Up to 50 files at a time, with a 25 MB frontend limit per file. |
| Analysis batch | Up to 50 distinct stored documents. |
| Active batches | Up to three admitted batches at a time. |
| Analysis execution | Two worker threads and four admitted analysis jobs; batches feed this same queue. |
| Full queue | Pending batch items wait for capacity. Direct job admission can return HTTP 429. |
| Duplicate submission | A repeated batch request identifier with the same payload returns the existing batch; a conflicting payload is rejected. |

## Partial failures and recovery

One failed document does not remove successful results from the batch. The batch can finish as completed with errors, and each failed item retains its message. Refreshing the browser reloads saved batch history and does not automatically create a second batch.

After a backend restart, batch items that were still pending can be scheduled. Analyses already admitted when interrupted are marked as errors under the existing job recovery policy. They require an explicit retry; the application does not claim to resume unfinished model inference.

Sources in unfinished batches cannot be deleted. This protects files needed by later items and makes a batch's lifetime separate from whether its browser tab remains open.

*Source basis S1 S4 S10*

# Document chat and local retrieval

Document Chat lets a user ask questions across selected files and continue with follow-ups. It works from extracted text. Direct visual understanding of images remains part of document analysis in Workbench; chat needs readable text or OCR output from the selected files.

## How an answer is produced

1. The user selects between one and 20 stored documents and enters a question.
2. The backend validates the selection and pins those sources against deletion during the request.
3. Readable pages are divided into overlapping passages. The default backend computes local lexical hash vectors; a configured provider encoder can be selected instead.
4. The current question and limited prior user context retrieve relevant passages from the selected sources. The service supplies up to six positive-similarity passages to generation.
5. The configured local reasoning model receives the question, limited conversation context, and numbered source passages. The prompt treats document contents as untrusted data and asks for an evidence-limited answer.
6. The response is checked for known numeric source markers. If markers are missing, invalid, or absent from a response paragraph, the application returns an insufficient-evidence message rather than displaying that answer.
7. Accepted markers open highlighted source previews. The user can inspect the actual cited passage and original.

## Search and generation have different roles

Knowledge Base search returns similar source passages. Chat uses retrieved passages to ask a local model for an answer. Embeddings are numerical representations used to compare passages with a query; they do not prove a statement is true. The default lexical method needs no learned embedding model, while a provider embedding configuration requires its local encoder to work.

## Conversation boundaries

The frontend keeps the conversation in memory in the current tab. Refreshing the page clears it, and changing sources starts a new conversation. A bounded recent history is sent for follow-ups. Conversation text helps interpret a question and is not treated as source evidence.

Only one chat generation is admitted at a time. A missing model produces an explicit availability error. No retrieved evidence produces an insufficient-evidence response without inventing an answer.

## What citation checking does not establish

The acceptance check validates marker presence and allowed source numbers. It does not establish that every claim logically follows from the passage. A model can still misinterpret a source, and OCR or retrieval can miss material. The website therefore presents answers as advisory and keeps the evidence accessible.

*Source basis S1 S5 S6 S8*

# Data storage and API organization

## Persistent and temporary state

| Data | Default location or mechanism | Lifetime |
|---|---|---|
| Uploaded originals | `data/uploads/` | Remain until explicitly deleted or managed externally. |
| Generated reports | `generated/` | Retained separately from deleted source files. |
| Structured records | `backend/kavach.db` using SQLite | Persists document metadata, tasks, runs, findings, evidence, reviews, audit entries, jobs, and batches. |
| Retrieval vectors | In-memory local index | Rebuilt from stored readable documents when the source signature changes or the process restarts. |
| Chat conversation | Current browser tab | Clears on page refresh or source selection change. |
| Sandbox working files | Disposable container storage | Discarded after the explicit code execution. |

Paths and providers can be changed through configuration. A local backend can retain documents and reports without a cloud database. The in-memory retrieval index should not be confused with a separately deployed persistent vector database.

## Representative API groups

| Endpoint group | Purpose |
|---|---|
| `/api/health` and `/api/system/status` | Service identity, configured mode, hardware, and service state. |
| `/api/documents` and `/api/documents/upload` | Stored sources and uploads; individual routes also provide previews, originals, deletion, and analysis. |
| `/api/analysis/jobs` | Admit an asynchronous document analysis and inspect its durable progress. |
| `/api/analysis/batches` | Admit or list batches and retrieve per-document outcomes. |
| `/api/knowledge` and `/api/knowledge/search` | Index information and local passage search. |
| `/api/chat` | Questions and bounded follow-up context scoped to selected document IDs. |
| `/api/evidence/{id}/preview` | Resolve a saved finding's excerpt and source highlight. |
| `/api/agents/runs` and `/api/review` | Analysis history, execution details, and human decisions. |
| `/api/deliverables` and `/api/sandbox` | Report downloads and explicitly requested code execution. |

## State and error semantics

The backend validates payloads with Pydantic and returns explicit errors for missing documents, unavailable sources or models, conflicting operations, invalid settings, and full queues. Jobs and batch items keep their own statuses so a browser disconnect is not treated as a reason to start duplicate work. Older synchronous analysis endpoints remain available for compatibility.

*Source basis S1 S4 S5 S10*

# Local setup and daily operation

## Required services and optional capabilities

The repository's setup guidance uses Python 3.10 or later and Node.js 18 or later. Install the frontend and backend dependencies, provision the configured local models, and run the backend and frontend. Available hardware and model size affect practical inference speed and memory use; this report does not claim a universal hardware minimum or response time.

Tesseract must be installed separately for local OCR. PDF text extraction, scanned-page rendering, Word reading, and additional report formats depend on their optional packages. Installing core API dependencies alone does not provide every analysis or export capability.

## Typical startup commands

Run the backend from the `backend` directory:

```
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

For the already configured native sandbox setup, the environment file can be loaded explicitly:

```
python -m uvicorn app.main:app --env-file .env.sandbox --host 127.0.0.1 --port 8000
```

Run the frontend from the `frontend` directory:

```
npm run dev
```

For a production frontend build, use `npm run build`, followed by `npm run start`. The default website URL is `http://localhost:3000`; the local backend uses port 8000. Build-time `NEXT_PUBLIC_API_BASE` must point to the backend that the browser can reach. Run a single backend process for the current workstation queue design.

## Checking readiness

Use System to inspect available service and hardware readings. Use Model Router to inspect assignments, then verify actual inference through an analysis or chat request. Confirm that required model files are already provisioned for offline operation. An unavailable backend or model should be corrected rather than interpreted as an empty analytical result.

## Common operating situations

- A scan returns no readable text: check OCR availability and scan quality.
- A model request fails: check the configured provider, model files, and local runtime.
- A direct job receives HTTP 429: wait for capacity in the local queue.
- A file cannot be deleted: wait for its active analysis, batch, or chat request to finish.
- A citation cannot be opened: confirm that the original still exists and matches the saved excerpt.
- A report format is absent: use an available format or provision the optional exporter.

*Source basis S1 S8 S9 S10 S12*

# Security boundaries and current limitations

## Implemented application controls

Uploads are restricted by extension and size, filenames are sanitized, and document serving checks source paths. Airgapped provider policy blocks configured external providers and limits local inference URLs to loopback hosts. Planner actions are allowlisted and have bounded execution. Source pinning prevents deletion during active operations, and activity records are stored locally.

The Python Sandbox is separate from document analysis. Its fixed policy disables container networking, avoids host mounts, uses a read-only root filesystem, runs the untrusted worker as a non-root user, and limits resources. Defaults include one CPU, 128 MB RAM, 32 processes, ten seconds, 64 KB combined output, and a disposable 16 MB temporary area. Its runtime must be provisioned separately, and shared-kernel isolation still depends on a maintained host.

## Deployment limits

The current system is intended for a trusted local workstation. Application policy is not a substitute for authentication, authorization, transport security, disk encryption, backups, or host-level egress restrictions. User accounts, role-based multi-user isolation, and a complete collaboration workflow are not implemented. Wider exposure requires those controls to be designed and tested.

## Analysis and retrieval limits

- Risk review finds configured source-term patterns. It is not a comprehensive semantic or regulatory risk assessment.
- Field extraction targets supported literal formats. OCR mistakes, unusual layouts, languages, and formatting can affect results.
- Summaries and visual descriptions can omit or misinterpret information.
- A known citation marker is not a proof that the cited passage supports every generated claim.
- The default lexical index has different retrieval behavior from a learned semantic encoder.
- Chat history is not persisted across refreshes, and chat does not perform independent visual image reasoning.
- Restart recovery preserves status and pending batch work; it does not resume interrupted model inference.

## Further improvements

Practical next enhancements include user accounts and roles, document version comparison, collaborative reviewer comments and assignments, persistent chat sessions with explicit retention controls, backup and recovery, and a quality dashboard based on labeled evaluation data. These are future directions. Evidence highlighting, batch processing, and source-cited document chat are already implemented.

*Source basis S1 S3 S4 S5 S7 S9*

# Demonstration plan and recorded verification

## Suggested demonstration sequence

1. Explain the home page: the project helps review private documents with source evidence and local processing.
2. Upload an invoice or text record in Document Vault. Show the upload outcome and original preview.
3. In Workbench, select the finance profile and Extract fields. Run the analysis and explain the real stage progress.
4. Open a field's highlighted source, compare it with the original, and show its saved run and available report downloads.
5. Upload or select a second source and start a batch. Show separate item outcomes and explain that one failure does not discard successful work.
6. Open Document Chat, select the relevant files, ask a question, and follow it with a question referring to the previous answer. Open a citation.
7. Search the Knowledge Base, then briefly show Model Router, Security Center, and System. Demonstrate the Python Sandbox separately if its runtime is ready.

Use documents that contain the details being demonstrated. Avoid describing an empty match list as proof that the input is safe or complete.

## Recorded implementation checks

| Check | Recorded result | What it establishes |
|---|---|---|
| Targeted backend regression run | 111 tests passed in the implementation session. | Covered features and related document, OCR, planner, persistence, and failure paths under test conditions. |
| Frontend build | Next.js production build and type checking passed. | The updated frontend compiled, including the chat route and batch and evidence components. |
| Local HTTP checks | Updated pages and new batch and chat API routes responded as expected. | The running frontend and backend exposed the integrated routes. |
| Real local inference smoke check | Two fixture sources completed as a batch; four source highlights matched; Qwen3 8B answered and handled a follow-up with citations. | The real configured model participated in an isolated end-to-end example. |

The recorded smoke source stated that a supplier agreement's renewal deadline was 2030-01-12. The real model returned that date with a source marker and correctly connected a follow-up question to the renewal deadline. This was synthetic test content, not a user or customer record.

These checks are implementation evidence, not a benchmark of general document accuracy. Most unit tests isolate inference with test-only doubles. The separate smoke check used actual Ollama inference. Browser interaction and visual UI checks were unavailable in the implementation environment; this report does not claim they were completed. The checks above were recorded during implementation, not rerun merely to write this report.

*Source basis S11*

# Glossary and implementation references

## Essential terms

| Term | Meaning in KAVACH AI |
|---|---|
| OCR | Optical character recognition converts scanned or photographed text into readable text. |
| Local inference | AI generation performed by the configured local model runtime. |
| Profile | Domain context and vocabulary used for a document task. |
| Analysis mode | The requested operation: risk review, summary, or field extraction. |
| Agent and planner | A software component with a defined role; the optional planner chooses among allowed actions. |
| RAG and embedding | Retrieval-augmented generation supplies source passages to a model; embeddings represent text for similarity search. |
| Evidence and verification | A source excerpt and location, plus checks of citation support. |
| Job and batch | A persisted document analysis; a group of documents scheduled through the same bounded queue. |
| Sandbox | A restricted, temporary environment for code explicitly submitted by a user. |

## Source index

All paths below are relative to the project root. They identify implementation sources used for this report; they are not external product endorsements.

S1 `README.md` provides setup, feature behavior, operating scope, and deployment qualifications.

S2 `frontend/app/` and `frontend/components/shell/nav.ts` define the website pages and navigation.

S3 `backend/app/services/analysis.py` and `document_analysis.py` implement extraction and analysis behavior.

S4 `backend/app/services/analysis_jobs.py` and `batch_jobs.py` implement queues, durable progress, and recovery.

S5 `backend/app/services/document_chat.py`, `knowledge.py`, and `backend/app/rag/pipeline.py` implement scoped chat and retrieval.

S6 `backend/app/services/evidence_preview.py`, `document_store.py`, and `frontend/components/vault/` implement source access and highlighting.

S7 `backend/app/agents/` contains orchestration, planner actions, progress, tools, and human review rules.

S8 `backend/app/models/` and `backend/app/config.py` define local providers, routing, and defaults.

S9 `backend/app/security/` and `docker/sandbox/` define application guards and sandbox policy.

S10 `backend/app/database/db.py`, `api/routes.py`, `schemas/schemas.py`, and `services/deliverables.py` define persistence, APIs, validation, and reports.

S11 `backend/tests/test_document_features.py`, related regression tests, and `backend/scripts/verify_document_features.py` support validation. Recorded real-model evidence is in `tmp/document-features-gr9fk5na/result.json`.

S12 `frontend/package.json` and `backend/requirements.txt` identify the implemented stack and dependencies.

## Project explanation in one paragraph

KAVACH AI is a local document intelligence workbench. It reads uploaded documents, uses selected analysis modes and configured local models, retrieves relevant passages, and helps users inspect source-supported results. Users can review findings, highlight evidence, process batches, ask document questions, and download generated reports while retaining control over their local workflow.